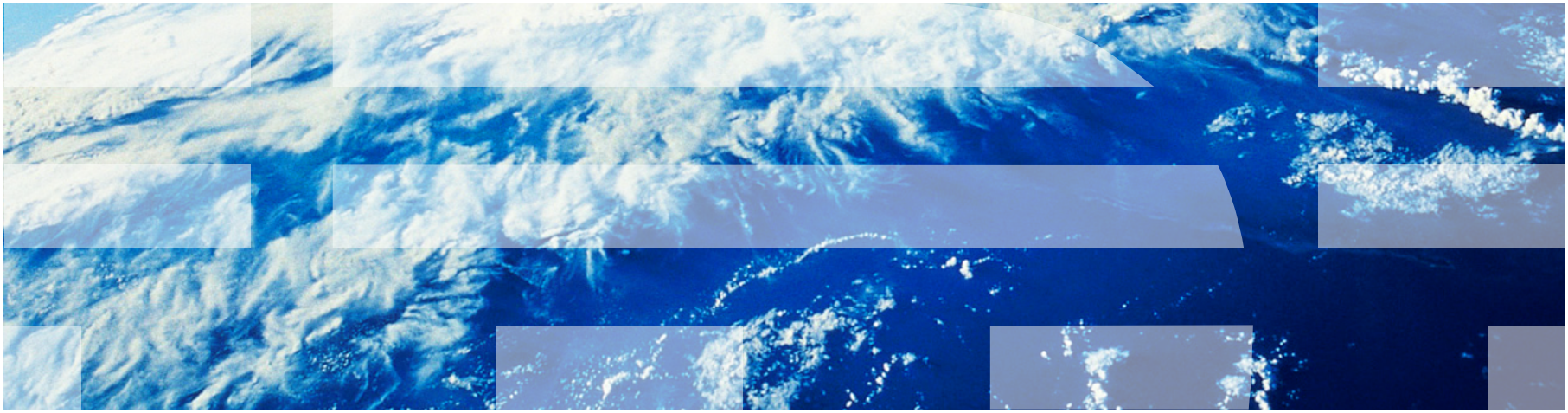
Motivation: Concurrent Transactions



Back to our bank example

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@4:29 am day+1)

Account	....	Balance ($)
3001		550
4001		110
5001		22
6001		66
3002		88
4002		-220
5002		352
...		
30108		-110
40008		110
50002		22

Other Transactions

10:02 am Acct 3001: Wants 600$
11:45 am Acct 5002: Wire for 1000$
. . . . .
. . . . .
2:02 pm Acct 3001: Debit card for $12.37

'T-Monthly-423'

Monthly Interest 10%
4:28 am Starts run on 10M bank accounts
Takes 24 hours to run

```
UPDATE Money
SET Balance = Balance * 1.1
```

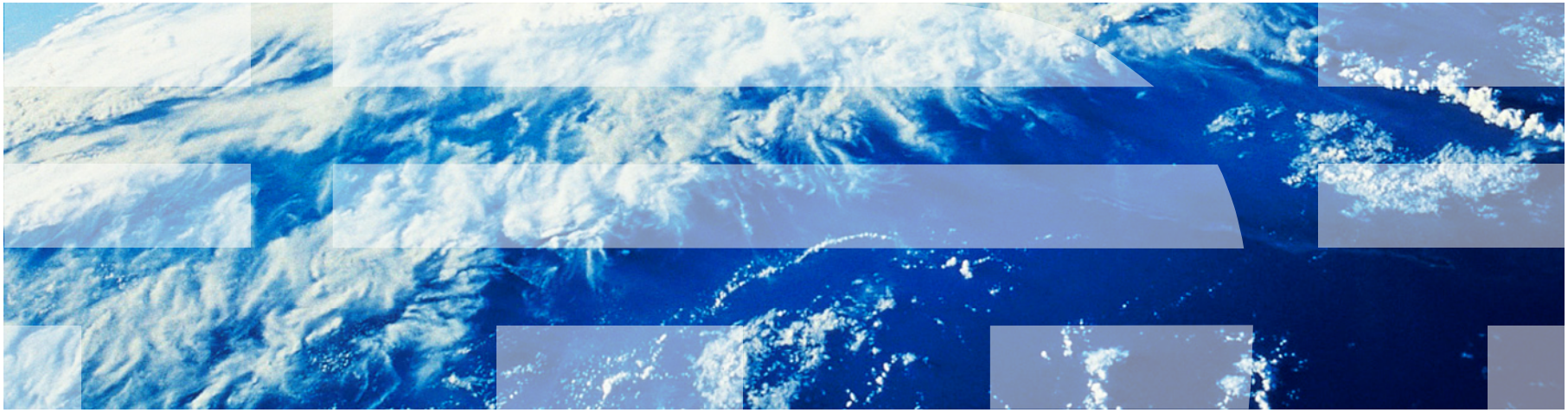
Q: How do I not wait for a day to access my $$$s?

Big idea: locks

- Intuition
  - ”Lock” each record for shortest time possible

- Key questions
  - Which records?
  - For how long?
  - What is the algorithm for holding them?

Concurrency, Scheduling and Anomalies



Concurrency: Isolation & Consistency

- DB is responsible for concurrency so that…

Isolation is maintained: Users must be able to execute each TXN as if they were the only user

ACID

Consistency is maintained: TXNs must leave the DB in consistent state

ACID

Example- consider two TXNs:

```
T1: START TRANSACTION
            UPDATE Accounts
            SET Amt = Amt + 100
            WHERE Name = 'A'

            UPDATE Accounts
            SET Amt = Amt - 100
            WHERE Name = 'B'
COMMIT
```

T1 transfers $100 from B's account to A's account

```
T2: START TRANSACTION
            UPDATE Accounts
            SET Amt = Amt * 1.06
COMMIT
```

T2 credits both accounts with a 6% interest payment

Note:

1. DB does not care if T1 —> T2 or T2 —> T1 (which TXN executes first)
2. If developer does, what can they do? (Put T1 and T2 inside 1 TXN)

Example

T_1 | A += 100 | B -= 100

T1 transfers $100 from B's account to A's account

T_2 | A *= 1.06 | B *= 1.06

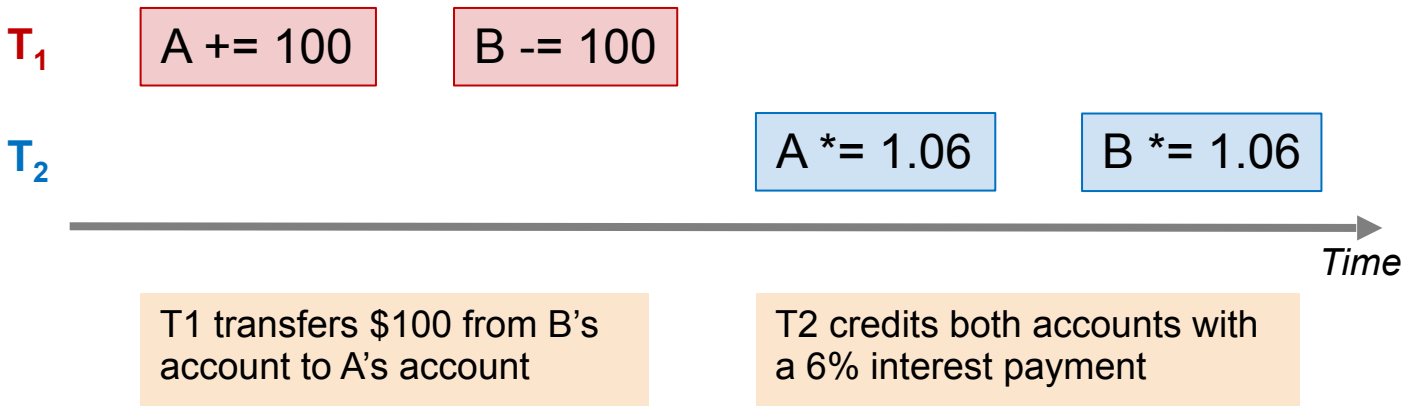
T2 credits both accounts with a 6% interest payment

Goal for scheduling transactions:

- Interleave transactions to boost performance
- Data stays in a good state after commits and/or aborts (ACID)

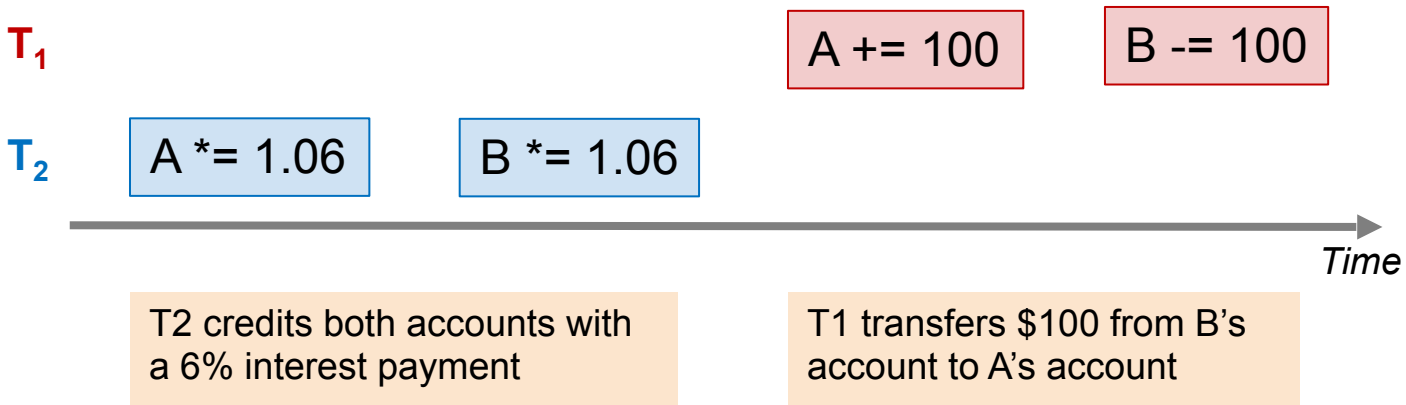
Example- consider two TXNs:

We can look at the TXNs in a timeline view- serial execution:



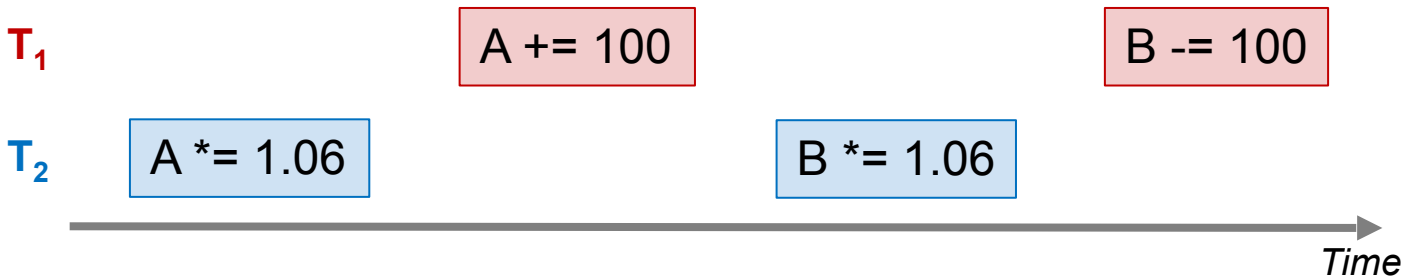
Example- consider two TXNs:

The TXNs could occur in either order… DB allows!



Example- consider two TXNs:

The DB can also **interleave** the TXNs



T2 credits A's account with 6% interest payment, then T1 transfers $100 to A's account…

T2 credits B's account with a 6% interest payment, then T1 transfers $100 from B's account…

Interleaving & Isolation

- The DB has freedom to interleave TXNs
- However, it must pick an interleaving or schedule such that isolation and consistency are maintained
- $\Rightarrow$ Must be *as if* the TXNs had executed serially!

DB must pick a schedule which maintains isolation & consistency

Scheduling examples

Starting Balance

A	B
$50	$200

Serial schedule T_1,T_2:

T_1: A += 100 | B -= 100

T_2: A *= 1.06 | B *= 1.06

A	B
$159	$106

Interleaved schedule A:

T_1: A += 100 | B -= 100

T_2: A *= 1.06 | B *= 1.06

A	B
$159	$106

Same result!

Scheduling examples

Starting Balance

A	B
$50	$200

Serial schedule T_1,T_2:

T_1: A += 100 | B -= 100

T_2: A *= 1.06 | B *= 1.06

A	B
$159	$106

Interleaved schedule B:

T_1: A += 100 | B -= 100

T_2: A *= 1.06 | B *= 1.06

A	B
$159	$112

Different result than serial T_1,T_2!